

Semana 9: Colecciones — Listas, diccionarios y la estructura de los datos

Guía completa de la sesión

Visión general

Duración total	3 horas (1.5 h cátedra con live coding + 1.5 h laboratorio en Colab)
Objetivo central	Que los estudiantes manejen listas y diccionarios como estructuras para organizar datos ecológicos, y las combinen con bucles y condicionales para análisis básico
Idea ancla	En la Semana 8, una variable guardaba un dato. Pero un inventario forestal tiene cientos de árboles, cada uno con especie, DAP, altura. Necesitamos estructuras que organicen muchos datos juntos — y eso son las colecciones
Prerrequisito	Semana 8 (variables, tipos, operaciones, if/elif/else, for, while)
Materiales	Proyector, Colab, WiFi, notebook starter, Dataset 1 (“Bosque Valdiviano Simple”) como CSV
Conexión con Sección 1	Sem. 3: la tabla de frecuencias de especies → diccionario. Sem. 4: recorrer una lista ordenándola → for sobre lista

PARTE 1: CÁTEDRA CON LIVE CODING (90 minutos)

Bloque A — ¿Por qué colecciones? (5 min)

Problema concreto:

```
# Semana 8: una variable por dato
especie_1 = "N. dombeyi"
especie_2 = "D. winteri"
especie_3 = "A. luma"
# ... ¿y si hay 200 especies?
```

“No van a crear 200 variables. Necesitan una estructura que contenga muchos datos bajo un solo nombre. Eso es una lista.”

Bloque B — Listas: la colección ordenada (30 min)

Crear y acceder (8 min)

```
# Crear una lista
especies = ["N. dombeyi", "D. winteri", "A. luma", "E. cordifolia", "L. philippiana"]
abundancias = [45, 28, 67, 15, 33]
alturas = [22.1, 15.8, 12.3, 25.4, 18.7]

print(especies)
print(len(especies))          # 5 elementos

# Indexación (empieza en 0 - como strings, como celdas de memoria)
print(especies[0])            # 'N. dombeyi'
print(especies[2])            # 'A. luma'
print(especies[-1])           # 'L. philippiana'

# Slicing
print(especies[1:3])          # ['D. winteri', 'A. luma']
```

```
print(especies[:3])           # los primeros 3
print(especies[2:])           # del tercero al final
```

“El índice funciona igual que en strings (Semana 8) y que en las celdas de memoria (Semana 1). Posición 0 = primer elemento.”

Modificar listas (8 min)

```
# Las listas SÍ son mutables (a diferencia de los strings)
especies[0] = "Nothofagus dombeyi"      # cambiar un elemento
print(especies)

# Agregar elementos
especies.append("Fitzroya cupressoides") # al final
print(especies)
print(len(especies))                    # 6

# Insertar en una posición específica
especies.insert(0, "Araucaria araucana") # al inicio
print(especies)

# Eliminar
especies.remove("D. winteri")           # por valor
print(especies)

ultimo = especies.pop()                  # elimina y devuelve el último
print(f"Eliminado: {ultimo}")
print(especies)

# Verificar si un elemento está en la lista
print("A. luma" in especies)             # True
print("Sequoia sempervirens" in especies) # False
```

Métodos útiles y funciones built-in (7 min)

```
numeros = [45, 28, 67, 15, 33, 67, 12]

print(len(numeros))           # 7
print(sum(numeros))           # 267
print(min(numeros))           # 12
print(max(numeros))           # 67
print(sorted(numeros))         # [12, 15, 28, 33, 45, 67, 67] - nueva lista ordenada
print(numeros)                 # [45, 28, ...] - la original NO cambió

# .sort() modifica la lista original (in-place)
numeros.sort()
print(numeros)                 # [12, 15, 28, 33, 45, 67, 67]

# .sort(reverse=True) - orden descendente
numeros.sort(reverse=True)
print(numeros)                 # [67, 67, 45, 33, 28, 15, 12]

# .count() - contar ocurrencias
print(numeros.count(67))       # 2

# .index() - encontrar la posición
print(numeros.index(33))       # 3
```

Callback Sem. 4: “¿Recuerdan el bubble sort? Python usa `list.sort()`, que internamente usa **Timsort** — un algoritmo $O(n \log n)$. Eligió el eficiente por ustedes.”

Recorrer listas con for (7 min)

```
# Recorrer elementos directamente
for especie in especies:
    print(f" - {especie}")

# Recorrer con índice (cuando necesitas la posición)
for i in range(len(abundancias)):
    print(f" Especie {i+1}: {abundancias[i]} individuos")

# Mejor: enumerate() - ambos a la vez
for i, especie in enumerate(especies):
    print(f" {i+1}. {especie}")
```

Construir listas con bucles (patrones comunes) (5 min — si hay tiempo)

```
# Patrón 1: lista vacía + append
proporciones = []
total = sum(abundancias)
for n in abundancias:
    proporciones.append(n / total)
print(proporciones)

# Patrón 2: list comprehension (versión compacta)
proporciones = [n / total for n in abundancias]
print(proporciones)

# Patrón 3: filtrar con list comprehension
grandes = [n for n in abundancias if n > 30]
print(f"Abundancias > 30: {grandes}")
```

“La list comprehension es un atajo elegante. Dice: ‘para cada n en abundancias, calcula n/total y ponlo en la nueva lista.’ Es un for comprimido en una línea.”

Bloque C — Diccionarios: la tabla de frecuencias (25 min)

Callback Sem. 3: “En la Semana 3, hicieron tablas de frecuencias: especie $\rightarrow$ abundancia. Un diccionario es exactamente eso: pares de clave $\rightarrow$ valor.”

Crear y acceder (8 min)

```
# Un diccionario asocia CLAVES con VALORES
inventario = {
    "N. dombeyi": 45,
    "D. winteri": 28,
    "A. luma": 67,
    "E. cordifolia": 15,
    "L. philippiana": 33
}

print(inventario)
print(inventario["A. luma"])      # 67 - acceso por clave
print(inventario.get("Pudú", 0))  # 0 - .get() con valor por defecto

# Claves y valores
print(inventario.keys())          # las especies
print(inventario.values())        # las abundancias
print(len(inventario))            # 5 especies
```

“En una lista, acceden por posición (índice numérico). En un diccionario, acceden por nombre (clave). Es como la diferencia entre ‘celda 3’ y ‘celda llamada abundancia_luma’.”

Modificar diccionarios (5 min)

```
# Agregar una especie
inventario["F. cupressoides"] = 3
print(len(inventario))          # 6

# Modificar un valor
inventario["N. dombeyi"] = 48   # corregir el conteo
print(inventario["N. dombeyi"]) # 48

# Eliminar
del inventario["F. cupressoides"]

# Verificar si una clave existe
print("A. luma" in inventario)   # True
print("Sequoia" in inventario)  # False
```

Recorrer diccionarios (7 min)

```
# Recorrer claves
for especie in inventario:
    print(especie)

# Recorrer claves y valores juntos
for especie, abundancia in inventario.items():
    print(f" {especie}: {abundancia} individuos")

# Calcular total y proporciones
total = sum(inventario.values())
print(f"\nTotal: {total} individuos\n")

for especie, n in inventario.items():
    p = n / total
    print(f" {especie}: {p:.3f} ({p*100:.1f}%)")
```

Construir diccionarios desde datos (5 min)

```
# Patrón: contar ocurrencias (frecuencia)
observaciones = ["coigüe", "luma", "canelo", "luma", "coigüe",
                 "luma", "coigüe", "canelo", "luma", "ulmo"]

conteo = {}
for obs in observaciones:
    if obs in conteo:
        conteo[obs] += 1
    else:
        conteo[obs] = 1

print(conteo)
# {'coigüe': 3, 'luma': 4, 'canelo': 2, 'ulmo': 1}
```

“Esto es exactamente la tabla de frecuencias de la Semana 3 — pero construida automáticamente. Ya no necesitan contar a mano.”

```
# Versión más limpia con .get()
conteo = {}
for obs in observaciones:
    conteo[obs] = conteo.get(obs, 0) + 1

print(conteo)
```

Bloque D — Tuplas: la colección inmutable (5 min)

```
# Una tupla es como una lista, pero NO se puede modificar
coordenadas = (-39.81, -73.24)
print(coordenadas[0])      # -39.81
print(coordenadas[1])      # -73.24

# Intentar modificar → error
# coordenadas[0] = -40.0    # TypeError: 'tuple' does not support item assignment

# Desempaquetar
latitud, longitud = coordenadas
print(f"Lat: {latitud}, Lon: {longitud}")
```

¿Cuándo usar qué?

Estructura	Mutable	Uso típico
Lista []	Sí	Secuencia de datos del mismo tipo (abundancias, DAPs)
Diccionario {}	Sí	Pares clave-valor (especie → abundancia)
Tupla ()	No	Datos que no deben cambiar (coordenadas, constantes)

Bloque E — Listas de diccionarios: la estructura de datos ecológicos (10 min)

“Un inventario forestal real tiene muchos árboles, cada uno con varias mediciones. ¿Cómo organizamos eso?”

```
# Cada árbol es un diccionario
arbol_1 = {"especie": "N. dombeyi", "dap": 45.3, "altura": 22.1, "parcela": 1}
arbol_2 = {"especie": "D. winteri", "dap": 25.1, "altura": 15.8, "parcela": 1}
arbol_3 = {"especie": "A. luma", "dap": 18.7, "altura": 12.3, "parcela": 2}

# El inventario es una lista de diccionarios
inventario = [arbol_1, arbol_2, arbol_3]

# Recorrer
for arbol in inventario:
    print(f"{arbol['especie']}: DAP={arbol['dap']} cm, H={arbol['altura']} m")

# Filtrar: árboles con DAP > 20
grandes = [a for a in inventario if a["dap"] > 20]
print(f"\n{len(grandes)} árboles con DAP > 20 cm:")
for a in grandes:
    print(f" {a['especie']} ({a['dap']} cm)")

# Calcular promedio de DAP
daps = [a["dap"] for a in inventario]
promedio_dap = sum(daps) / len(daps)
print(f"\nDAP promedio: {promedio_dap:.1f} cm")
```

“Esta estructura — lista de diccionarios — es la versión manual de lo que Pandas hará automáticamente en la Semana 13. Cada diccionario es una fila, cada clave es una columna.”

Bloque F — Programa integrador: análisis de diversidad (5 min)

```
import math

# Datos
```

```

inventario = {
    "N. dombeyi": 45, "D. winteri": 28, "A. luma": 67,
    "E. cordifolia": 15, "L. philippiana": 33,
    "A. araucana": 8, "F. cupressoides": 3
}

# Riqueza
S = len(inventario)

# Shannon H'
total = sum(inventario.values())
H = 0
for especie, n in inventario.items():
    p = n / total
    H -= p * math.log(p)

# Equitatividad
H_max = math.log(S)
J = H / H_max

# Reporte
print(f"Riqueza (S): {S} especies")
print(f"Abundancia total: {total} individuos")
print(f"Shannon H': {H:.4f} nats ({H/math.log(2):.4f} bits)")
print(f"H' máximo: {H_max:.4f} nats")
print(f"Equitatividad (J): {J:.4f}")

```

“En la Semana 3 esto tomaba 20 minutos con calculadora. Ahora toma 10 líneas y funciona para cualquier número de especies.”

PARTE 2: LABORATORIO EN COLAB (90 minutos)

Dataset: Bosque Valdiviano Simple

El laboratorio usa un CSV sintético con 100 observaciones de árboles. El CSV se carga desde una URL o se genera en el notebook.

Columnas: especie, dap_cm, altura_m, parcela (1–5) Especies: N. dombeyi, D. winteri, A. luma, E. cordifolia, L. philippiana

Ejercicios

Ejercicio 1: Listas fundamentales (15 min) Crear listas a partir de datos ecológicos. Indexar, hacer slicing, append, remove. Usar len, sum, min, max, sorted.

Ejercicio 2: Listas + for (15 min) Calcular el DAP promedio de una lista de 15 árboles con un bucle. Filtrar árboles con altura > 20m. Encontrar la especie con mayor abundancia.

Ejercicio 3: Diccionarios — tabla de frecuencias (15 min) Dada una lista de observaciones de especies (con repeticiones), construir un diccionario de conteos. Calcular la proporción de cada especie.

Ejercicio 4: Diccionarios — inventario ecológico (15 min) Crear un diccionario especie → abundancia. Recorrer con .items(). Encontrar la especie más abundante y la menos abundante.

Ejercicio 5: Listas de diccionarios (15 min) Trabajar con una lista de diccionarios (árboles con especie, DAP, altura, parcela). Filtrar por parcela. Calcular estadísticas por especie.

Ejercicio 6 (desafío): Diversidad por parcela (15 min) Calcular Shannon H' para cada una de las 5 parcelas por separado. ¿Cuál es la más diversa? Usar diccionarios anidados o funciones (adelanto de Sem. 11).

Tarea evaluada (primera de la Sección 2)

Entrega: Semana 10 (inicio de la clase) **Formato:** Notebook de Colab compartido **LLMs:** Permitidos con prompt log documentado en una celda al final

Enunciado: Se proporciona un CSV con registros de cámaras trampa simulados (especie, fecha, sitio, método). Los estudiantes deben:

1. Cargar los datos como lista de diccionarios (se proporciona el código de carga)
2. Contar cuántas observaciones hay por especie (diccionario de frecuencias)
3. Identificar la especie más y menos observada
4. Filtrar las observaciones de un sitio específico
5. Calcular Shannon H' para el dataset completo
6. (Bonus) Comparar H' entre dos sitios

Notas pedagógicas

La progresión lista → diccionario → lista de diccionarios

Esta es una escalera deliberada: - **Lista:** una columna de datos (las abundancias) - **Diccionario:** dos columnas vinculadas (especie → abundancia) - **Lista de diccionarios:** una tabla completa (cada fila = un diccionario)

La última estructura es la antesala de Pandas (Semana 13). Cuando vean `pd.DataFrame`, deberían pensar: “Ah, es una lista de diccionarios con superpoderes.”

La tabla de frecuencias como puente

El patrón “recorrer una lista y contar ocurrencias en un diccionario” es el puente más directo con la Semana 3 (entropía, frecuencias). Dedicarle tiempo — es un patrón que usarán constantemente.

Errores comunes de la Semana 9

Error	Causa	Solución
<code>IndexError: list index out of range</code>	Acceder a un índice que no existe	Verificar con <code>len()</code>
<code>KeyError: 'especie_x'</code>	Clave no existe en el diccionario	Usar <code>.get(clave, default)</code> o verificar con <code>in</code>
<code>TypeError: unhashable type: 'list'</code>	Usar una lista como clave de diccionario	Las claves deben ser inmutables (str, int, tuple)
Confundir <code>[]</code> y <code>{}</code>	Crear lista cuando querían diccionario o viceversa	<code>[]</code> = lista, <code>{}</code> = diccionario (o set vacío)
Modificar lista mientras se recorre	<code>for x in lista: lista.remove(x)</code>	Crear una nueva lista con list comprehension

Conexión con el resto del curso

Concepto de esta semana	Dónde se profundiza
Listas y métodos	Sem. 10 (listas como argumento de funciones), Sem. 13 (Series de Pandas)
Diccionarios	Sem. 11 (funciones que devuelven diccionarios), Sem. 13 (DataFrames)
List comprehension	Sem. 11 (dentro de funciones), Sem. 13 (Pandas <code>.apply()</code>)
Tabla de frecuencias con dict	Sem. 11 (función reutilizable), Sem. 13 (<code>.value_counts()</code>)
Lista de diccionarios	Sem. 13 (<code>pd.DataFrame(lista_de_dicts)</code> — la conversión directa)
CSV como fuente de datos	Sem. 12 (file I/O), Sem. 13 (<code>pd.read_csv()</code>)